MAHARAJA AGRASEN INSTITUTE OF TECHNOLOGY

RESTAURANT FOOD DELIVERY DATABASE SYSTEM

A MINI PROJECT REPORT
Submitted in partial fulfillment of DBMS

Submitted By:
Charu Sharma
Department of Information Technology

Under the Guidance of:
Ms. Vasudha

Academic Year:
2025–2026

Certificate

This is to certify that the DBMS mini project titled "**Online Learning Content Database with Versioning and Role-Based Access Control**" has been successfully completed by **Charu Sharma** in partial fulfillment of the requirements for the course Database Management System.

The work presented in this report is original and has been carried out under the guidance of the department. The project demonstrates the practical implementation of database concepts such as ER modeling, schema design, normalization, SQL queries, and referential integrity.

Date: 18 April

Place: Delhi

Signature of Examiner

Acknowledgement

I would like to express my sincere gratitude to the faculty members of the Department of Information Technology, Maharaja Agrasen Institute of Technology, for their valuable guidance and support throughout this DBMS mini project.

I am also thankful to my classmates and friends for their suggestions and encouragement. This project helped me strengthen my understanding of database design, SQL implementation, normalization, and the role of structured data management in real-world applications.

Finally, I would like to thank my family for their constant support and motivation.

CONTENT

- ☐ Introduction
- ☐ Problem Statement
- ☐ Objectives
- ☐ System Overview
- ☐ ER Diagram
- ☐ Schema
- ☐ Normalization
- ☐ Work Done
- ☐ SQL Queries
- ☐ Outputs
- ☐ Advantages
- ☐ Future Scope
- ☐ Conclusion

Introduction

The Restaurant Food Delivery Database System is designed to manage and organize the operations of an online food delivery platform.

In today's digital world, food delivery applications such as Zomato and Swiggy rely heavily on databases to manage customer orders, restaurant menus, and delivery processes.

This project focuses on building a structured database that stores and retrieves data efficiently. It demonstrates how DBMS concepts like entity relationships, normalization, and SQL queries are applied in real-life systems.

The system ensures accuracy, consistency, and easy access to data.

Problem Statement

Manual management of food delivery operations leads to various issues such as data redundancy, inconsistency, and difficulty in tracking orders.

Without a proper database system:

- Orders may get misplaced
- Customer data may be duplicated
- Payment tracking becomes difficult

This project aims to design a database that solves these issues by organizing data in a structured and efficient way.

Aim and Objectives

- The main objectives of this project are:
  - To design a relational database for a food delivery system
  - To maintain structured data for customers and restaurants
  - To track orders and delivery status
  - To implement SQL queries for data retrieval
  - To apply normalization techniques
  - To ensure data integrity and consistency

System Overview

The system consists of multiple entities that interact with each other.

Customers browse menus and place orders. Restaurants provide food items through menus. Each order may contain multiple items, which are stored in the Order_Items table.

The system also includes payment and delivery modules. Each order is linked with a payment record and delivery status.

This structured design helps in efficient data storage and retrieval.

Normalization

Normalization is the process of organizing data to reduce redundancy and improve data integrity.

1NF (First Normal Form):
All attributes contain atomic values and there are no repeating groups.

2NF (Second Normal Form):
All non-key attributes depend fully on the primary key.

3NF (Third Normal Form):
There are no transitive dependencies; all attributes depend only on the primary key.

By applying normalization, the database becomes efficient and avoids anomalies.

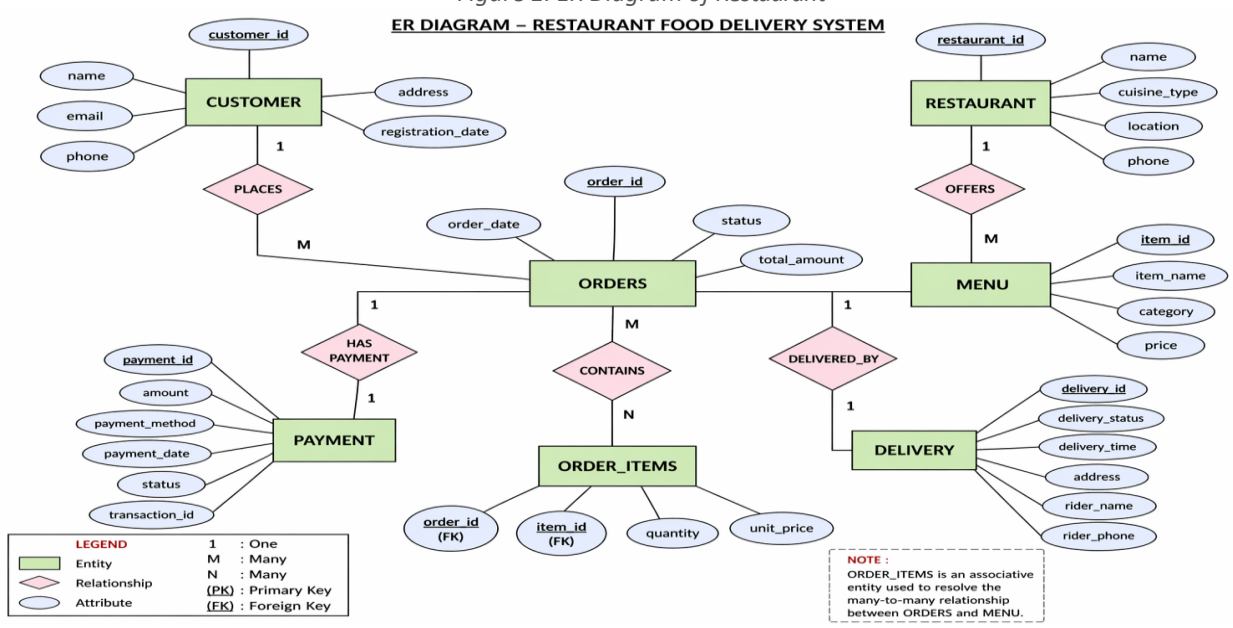
Entity-Relationship Diagram

The ER diagram shows the relationships between the main entities of the database.

The relationships between entities are defined as follows:

- A Customer can place multiple Orders (1:M relationship)
- A Restaurant can offer multiple Menu items (1:M relationship)
- An Order can contain multiple Order_Items (1:M relationship)
- Each Order_Item is linked to one Menu item (M:1 relationship)
- Each Order has one Payment (1:1 relationship)
- Each Order is associated with one Delivery (1:1 relationship)

Figure 1: ER Diagram of Restaurant



Relational Schema

📌 Schema Table

Table Name	Attributes	Primary Key	Foreign Key
Customer	customer_id, name, email, phone	customer_id	—
Restaurant	restaurant_id, name, location	restaurant_id	—
Menu	item_id, item_name, price, restaurant_id	item_id	restaurant_id → Restaurant
Orders	order_id, customer_id, order_date, status	order_id	customer_id → Customer
Order_Items	order_id, item_id, quantity	(order_id, item_id)	order_id → Orders, item_id → Menu
Payment	payment_id, order_id, amount, payment_method	payment_id	order_id → Orders
Delivery	delivery_id, order_id, delivery_status, delivery_time	delivery_id	order_id → Orders

Work Done

The project was implemented in multiple stages.

First, the requirements of the system were analyzed. Then, an ER diagram was created to represent the entities and relationships.

The relational schema was derived from the ER diagram. Tables were created using SQL commands in MySQL.

Sample data was inserted into each table to simulate real-world scenarios.

Queries were written to perform operations such as retrieving orders, calculating revenue, and joining tables.

The database was tested, and outputs were verified using SQL queries.

SQL Queries

---> Retrieve all orders:

```
SELECT * FROM Orders;
```

---> View menu items:

```
SELECT item_name, price FROM Menu;
```

---> Join multiple tables:

```
SELECT o.order_id, m.item_name, oi.quantity FROM Orders o JOIN Order_Items oi ON o.order_id = oi.order_id JOIN Menu m ON oi.item_id = m.item_id;
```

---> Calculate total revenue:

```
SELECT SUM(amount) FROM Payment;
```

---> Find most ordered item:

```
SELECT item_id, SUM(quantity) FROM Order_Items GROUP BY item_id ORDER BY SUM(quantity) DESC;
```

SQL Queries

---> Retrieve all orders:

```
SELECT * FROM Orders;
```

---> View menu items:

```
SELECT item_name, price FROM Menu;
```

---> Join multiple tables:

```
SELECT o.order_id, m.item_name, oi.quantity FROM Orders o JOIN Order_Items oi ON o.order_id = oi.order_id JOIN Menu m ON oi.item_id = m.item_id;
```

---> Calculate total revenue:

```
SELECT SUM(amount) FROM Payment;
```

---> Find most ordered item:

```
SELECT item_id, SUM(quantity) FROM Order_Items GROUP BY item_id ORDER BY SUM(quantity) DESC;
```

Output Screenshots

The following screenshots show the implementation and execution results of the project in My SQL Workbench.

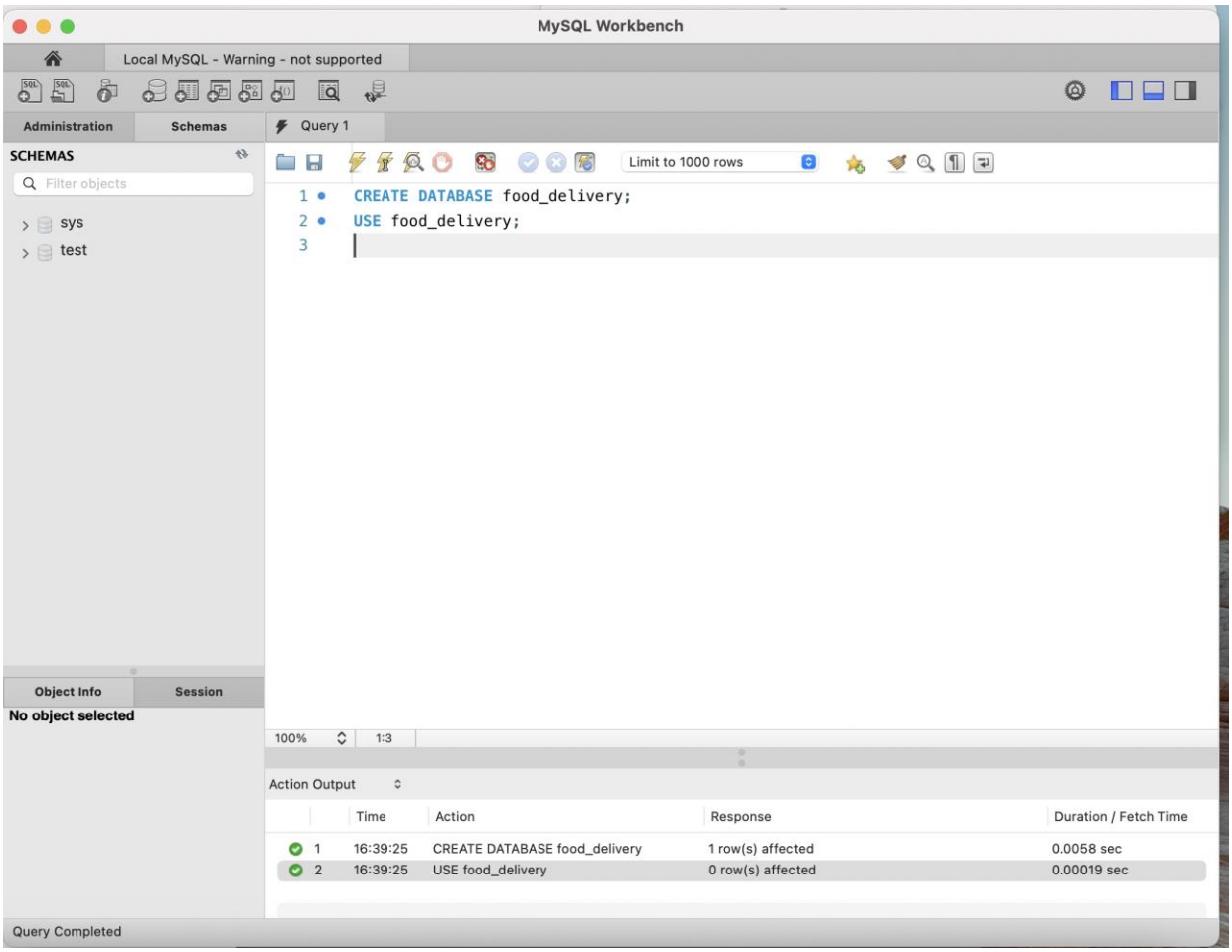


Figure 1: Database Creation

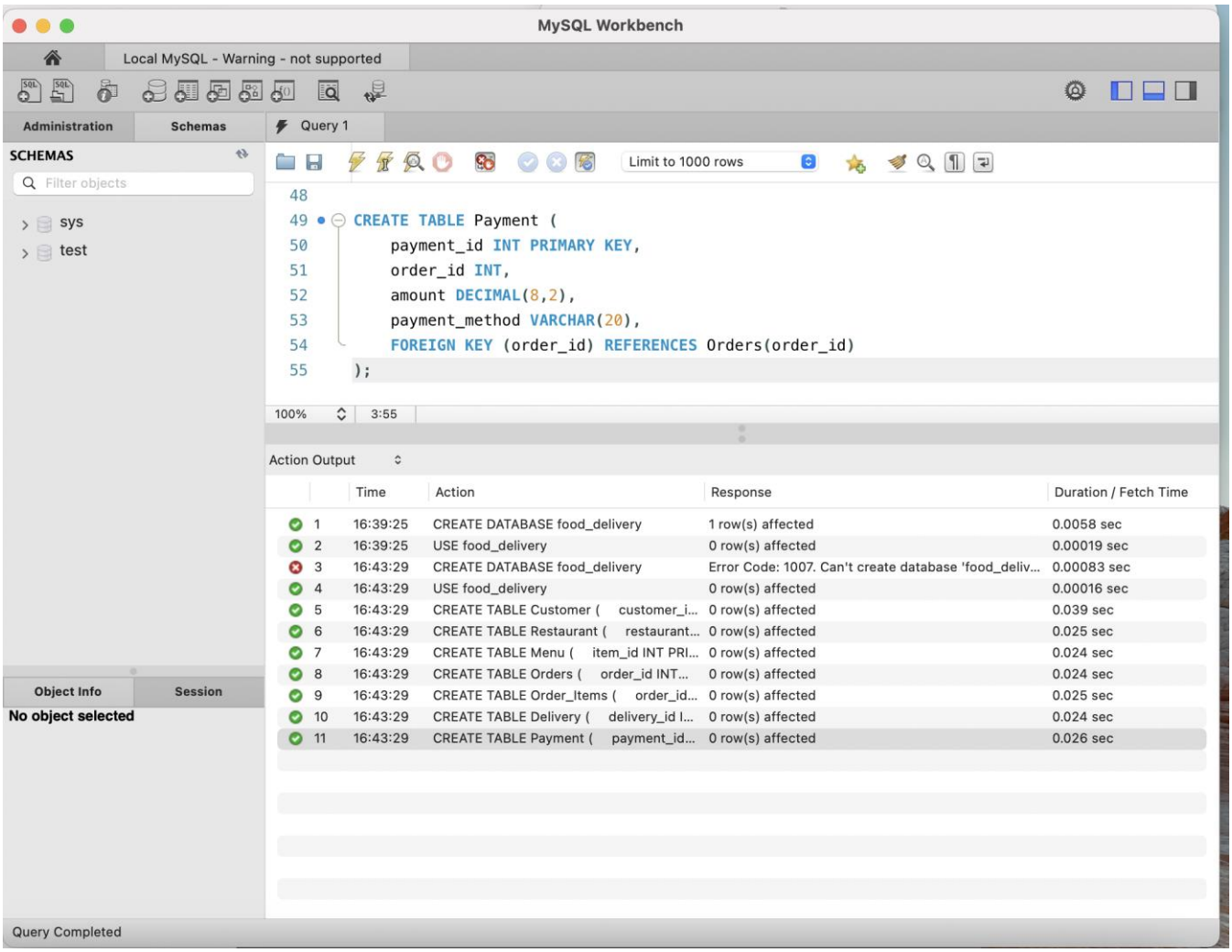
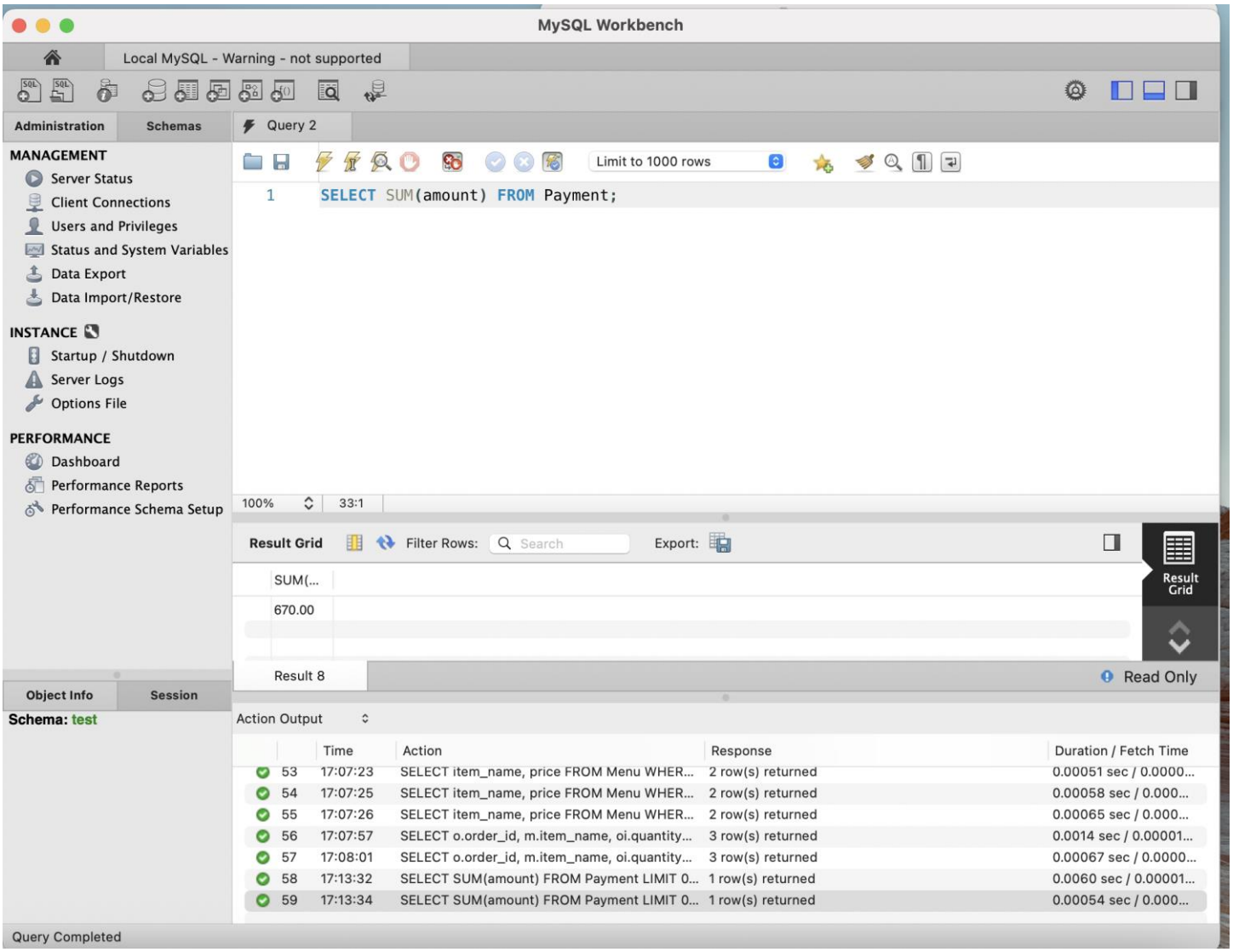


Figure 2: Table Creation


MySQL Workbench
Local MySQL - Warning - not supported
Administration
Schemas
Query 2
MANAGEMENT
Server Status
Client Connections
Users and Privileges
Status and System Variables
Data Export
Data Import/Restore
INSTANCE
Startup / Shutdown
Server Logs
Options File
PERFORMANCE
Dashboard
Performance Reports
Performance Schema Setup
Limit to 1000 rows
1 SELECT SUM(amount) FROM Payment;
100%
33:1
Result Grid
Filter Rows:
Search
Export:
SUM(...
670.00
Result Grid
Result 8
Read Only
Object Info
Session
Schema: test
Action Output
Time
Action
Response
Duration / Fetch Time
53 17:07:23 SELECT item_name, price FROM Menu WHER... 2 row(s) returned 0.00051 sec / 0.0000...
54 17:07:25 SELECT item_name, price FROM Menu WHER... 2 row(s) returned 0.00058 sec / 0.000...
55 17:07:26 SELECT item_name, price FROM Menu WHER... 2 row(s) returned 0.00065 sec / 0.000...
56 17:07:57 SELECT o.order_id, m.item_name, oi.quantity... 3 row(s) returned 0.0014 sec / 0.00001...
57 17:08:01 SELECT o.order_id, m.item_name, oi.quantity... 3 row(s) returned 0.00067 sec / 0.0000...
58 17:13:32 SELECT SUM(amount) FROM Payment LIMIT 0... 1 row(s) returned 0.0060 sec / 0.00001...
59 17:13:34 SELECT SUM(amount) FROM Payment LIMIT 0... 1 row(s) returned 0.00054 sec / 0.000...
Query Completed

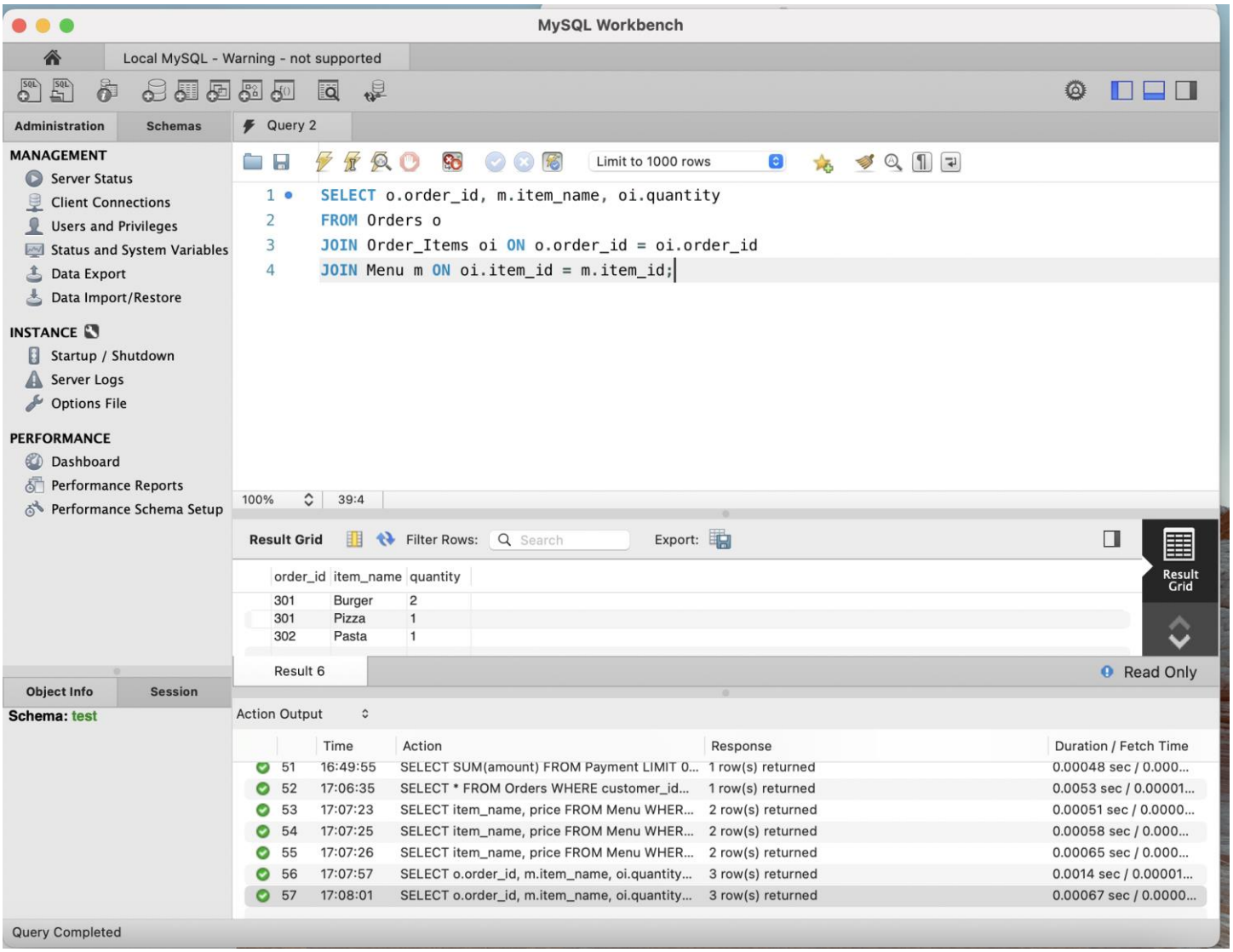


Figure 3 &4: JOIN & SELECT QUERIES

Advantages

The advantages of this system are:

- Efficient data management
- Easy tracking of orders
- Reduced redundancy through normalization
- Improved data consistency
- Scalable for real-world applications

Future Scope

The project can be enhanced by adding:

- Web-based interface
- Mobile application integration
- Online payment gateway
- User authentication system
- Real-time order tracking

Conclusion

The Restaurant Food Delivery Database System successfully demonstrates the use of DBMS concepts in a real-world application.

It provides a structured approach to managing data efficiently. The use of normalization and SQL queries ensures consistency and accuracy.

This project highlights the importance of database design in modern applications.

References

- MySQL Documentation official documentation
- DBMS lecture notes and class material
- Online learning platform database design concept